Parte 3: Laboratorio 3: Introducción a la Virtualización de Redes

Slicing de Red y Control de Flujos con POX

1. Introducción

En esta tercera práctica de laboratorio, exploraremos el concepto de virtualización de redes utilizando el controlador POX y Mininet. La virtualización de redes permite la abstracción de la red física, soportando múltiples redes lógicas que se ejecutan sobre un sustrato físico común compartido. Este enfoque es fundamental para la computación en la nube, redes multi-tenant y la experimentación en redes de producción [1, 2].

1.1. ¿Qué es la Virtualización de Redes?

La virtualización de redes es la capacidad de crear redes lógicas independientes sobre una misma infraestructura física. Cada red virtual actúa como un contenedor de servicios de red, permitiendo la coexistencia de múltiples arquitecturas y configuraciones sin interferencia mutua [3].

Los objetivos principales de diseño en la virtualización de redes incluyen:

- **Flexibilidad:** Soporte para diversas topologías y arquitecturas de enrutamiento.
- **Aislamiento:** Garantía de seguridad y separación de recursos entre redes lógicas.
- **Escalabilidad:** Maximización del número de redes virtuales coexistentes.
- **Programabilidad:** Capacidad de definir el comportamiento de la red mediante software.

1.2. Network Slicing

El *Network Slicing* (rebanado de red) es una técnica fundamental en la virtualización que consiste en dividir la red de producción en rebanadas"(*slices*) lógicas. Cada slice controla su propio reenvío de paquetes y recursos, asegurando un fuerte aislamiento: las acciones en un slice no afectan a otro.

Esto permite:

- **Múltiples inquilinos (Multi-tenancy):** Diferentes departamentos o clientes compartiendo la misma infraestructura.
- **Experimentación:** Ejecutar redes experimentales en paralelo con redes de producción.
- **Diferenciación de servicios:** Asignar recursos específicos (ancho de banda, latencia) a aplicaciones críticas.

1.3. FlowVisor

Una herramienta clave en este ecosistema es **FlowVisor**, que actúa como un proxy transparente entre los switches OpenFlow y múltiples controladores [4]. FlowVisor intercepta los mensajes OpenFlow y asegura que cada controlador solo pueda ver y modificar el tráfico perteneciente a su slice asignado, garantizando el aislamiento a nivel de plano de control. FlowVisor utiliza el concepto de "flowspace"para definir qué subconjunto de flujos puede controlar cada slice, mapeando dimensiones como direcciones MAC, IPs, puertos y VLANs a controladores específicos [4].

2. Preparación del entorno

Antes de iniciar, limpiaremos el entorno de Mininet para asegurar que no existan conflictos con ejecuciones previas.

```
mininet@mininet-vm:~$ sudo killall controller
mininet@mininet-vm:~$ sudo mn -c
```

3. Actividad 1: División por Capacidad (Slicing L2)

En esta primera actividad, dividiremos una topología simple en dos caminos aislados: uno de alta capacidad (100 Mbps) y otro de baja capacidad (10 Mbps). Los usuarios de un slice no podrán comunicarse con los del otro. Este ejercicio demuestra cómo SDN permite implementar aislamiento de tráfico y diferenciación de servicios mediante políticas de enrutamiento definidas por software [5].

3.1. Topología P3-1: Comprensión y Construcción

Utilizaremos la topología `p3-1` definida en `Topo.py`. Esta topología consiste en un anillo de 4 switches con enlaces redundantes de diferentes capacidades, diseñada específicamente para demostrar slicing basado en calidad de servicio (QoS).

Nota Importante: Corrección de la Topología Discrepancia entre código original y guías de laboratorio:
El archivo `Topo.py` original contenía una tercera opción de enlace definida como `linkopts3 = {'bw':1000}` (1000 Mbps). Sin embargo, las topologías mostradas en las guías oficiales del laboratorio (imágenes de referencia) **solo utilizan dos capacidades de enlace: 10 Mbps y 100 Mbps**.
Esta discrepancia ha sido corregida en esta guía paso a paso. El código presentado aquí elimina `linkopts3` y utiliza únicamente:

- `linkopts1 = {'bw':10}` para enlaces de baja capacidad
- `linkopts2 = {'bw':100}` para enlaces de alta capacidad

Corrección adicional importante: En el código original, los enlaces s1-h2 y s3-h4 estaban incorrectamente configurados con `linkopts[1]` (100 Mbps), lo que contradecía el objetivo del slicing por capacidad. Según las guías oficiales, estos hosts pertenecen al slice de baja velocidad y deben usar `linkopts[0]` (10 Mbps). Esta corrección asegura consistencia de ancho de banda extremo a extremo en cada slice:

Estructura de la topología P3-1:

La clase `P31(Topo)` en `Topo.py` implementa esta topología mediante el siguiente código:

```
class P31( Topo ):
  def __init__(self):
    Topo.__init__(self)
    linkopts1 = {'bw':10}
    linkopts2 = {'bw':100}
    linkopts = [linkopts1, linkopts2]

    switches = {}
    hosts = {}
    for i in range(1,5):
      sconfig = {'dpid': "%016x"% (i)}
      sn = self.addSwitch('s%s'% i,**sconfig)
      Host = self.addHost( 'h%s'% i,
        ip="10.0.0.%s/24"%i,
        mac="00:00:00:00:00:0%s"%i )
      switches['s%s'% i] = sn
      hosts['h%s'% i] = Host

    self.addLink('s1','s2',port1=1, port2=1,**linkopts[1])
    self.addLink('s2','s3',port1=2, port2=1,**linkopts[1])
    self.addLink('s1','s4',port1=2, port2=1,**linkopts[0])
    self.addLink('s3','s4',port1=2, port2=2,**linkopts[0])
    self.addLink('s1','h1',port1=3, port2=1,**linkopts[1])
    self.addLink('s1','h2',port1=4, port2=1,**linkopts[0])
    self.addLink('s3','h3',port1=3, port2=1,**linkopts[1])
    self.addLink('s3','h4',port1=4, port2=1,**linkopts[0])
```

Aspectos importantes de la construcción:

- **Asignación de DPID explícito**: Cada switch recibe un datapath ID único (`dpid`) en formato hexadecimal de 16 caracteres. Esto permite identificar switches específicos en el código del controlador.
- **Numeración de puertos específica**: Los parámetros `port1` y `port2` definen explícitamente qué puerto de cada switch se conecta. Esto es crucial para programar el reenvío correctamente en el controlador.
- **Dos caminos diferenciados**:
  - **Camino superior (100 Mbps):** Enlaces atravesando s1-s2-s3 para slice de alta velocidad
  - **Camino inferior (10 Mbps):** Enlaces atravesando s1-s4-s3 para slice de baja velocidad
- **Hosts conectados por capacidad**:
  - **Alta velocidad:** h1 (s1-puerto3, 100 Mbps) y h3 (s3-puerto3, 100 Mbps)
  - **Baja velocidad:** h2 (s1-puerto4, 10 Mbps) y h4 (s3-puerto4, 10 Mbps)
- **Configuración de ancho de banda**: Uso de Traffic Control (`TCLink`) mediante `linkopts` para limitar capacidad de cada enlace, emulando diferencias de QoS realistas.

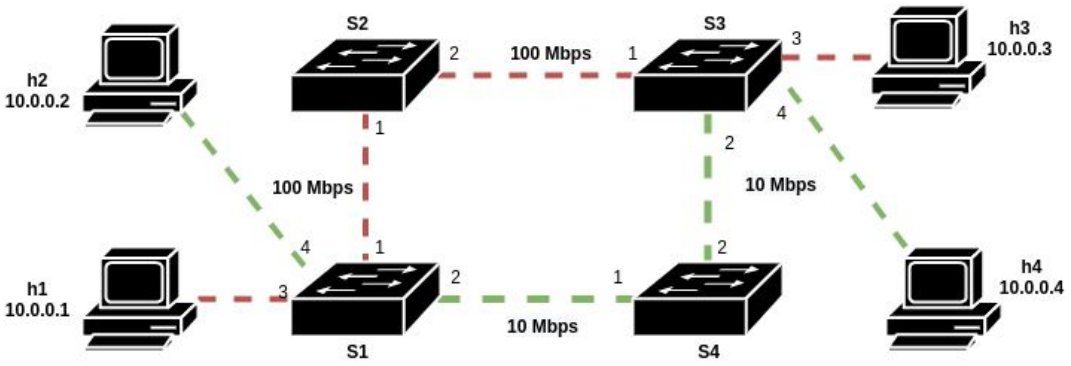


Figura 1: Topología P3-1: Dos caminos diferenciados por capacidad

Para iniciar la topología en Mininet:

```
mininet@mininet-vm:~$ sudo mn --custom Topo.py --topo p3-1
--controller remote,port=6633 --link tc
```

Explicación de parámetros del comando:

- `--custom Topo.py`: Carga el archivo de topología personalizado
- `--topo p3-1`: Selecciona la topología P31 del diccionario `topos`
- `--controller remote,port=6633`: Conecta a POX en puerto 6633
- `--link tc`: Habilita Traffic Control para aplicar restricciones de ancho de banda definidas en `linkopts`

3.2. Implementación del Slicing

El objetivo es configurar el controlador para que:

- El tráfico del Slice 1 (h1, h3) y (h2, h4) viaje por el camino de 100 Mbps
- El tráfico entre switches use los caminos de alta capacidad
- Los switches extreme (s1 y s3) reenvíen paquetes de hosts por rutas determinadas

Editaremos el archivo `topologySlice.py` para implementar la lógica en el evento `_handle_ConnectionUp`. Este evento se dispara cada vez que un switch se conecta al controlador, permitiéndonos instalar reglas de flujo proactivamente [6].

Lógica de programación del slicing:

1. Identificar cada switch por su DPID único
2. Para cada switch, definir reglas de reenvío estáticas basadas en puertos de entrada
3. Instalar reglas bidireccionales para cada conexión
4. No incluir acciones de aprendizaje dinámico (forwarding estático)

Código de solución (topologySlice.py):

```
def _handle_ConnectionUp(self, event):
  dpid = dpidToStr(event.dpid)
  log.debug("Switch %s has come up.", dpid)

  if dpid == "00-00-00-00-00-01":
    msg = of.ofp_flow_mod()
    msg.match.in_port = 3
    msg.actions.append(of.ofp_action_output(port=1))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 1
    msg.actions.append(of.ofp_action_output(port=3))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 4
    msg.actions.append(of.ofp_action_output(port=2))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 2
    msg.actions.append(of.ofp_action_output(port=4))
    event.connection.send(msg)

  elif dpid == "00-00-00-00-00-02":
    msg = of.ofp_flow_mod()
    msg.match.in_port = 1
    msg.actions.append(of.ofp_action_output(port=2))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 2
```

```
    msg.actions.append(of.ofp_action_output(port=1))
    event.connection.send(msg)

  elif dpid == "00-00-00-00-00-03":
    msg = of.ofp_flow_mod()
    msg.match.in_port = 3
    msg.actions.append(of.ofp_action_output(port=1))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 1
    msg.actions.append(of.ofp_action_output(port=3))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 4
    msg.actions.append(of.ofp_action_output(port=2))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 2
    msg.actions.append(of.ofp_action_output(port=4))
    event.connection.send(msg)

  elif dpid == "00-00-00-00-00-04":
    msg = of.ofp_flow_mod()
    msg.match.in_port = 1
    msg.actions.append(of.ofp_action_output(port=2))
    event.connection.send(msg)

    msg = of.ofp_flow_mod()
    msg.match.in_port = 2
    msg.actions.append(of.ofp_action_output(port=1))
    event.connection.send(msg)
```

Explicación del código:

- **Switch s1**: Tráfico desde h1 (puerto 3) usa puerto 1 hacia camino de 100 Mbps; tráfico desde h2 (puerto 4) usa puerto 2 hacia camino de 10 Mbps
- **Switch s2**: Relay bidireccional entre puertos 1 y 2 para el camino de 100 Mbps
- **Switch s3**: Simétrico a s1, distribuye tráfico de h3 y h4 según los caminos correspondientes
- **Switch s4**: Relay bidireccional entre puertos 1 y 2 para el camino de 10 Mbps

Ejecución del controlador:

```
mininet@mininet-vm:~$ cd ~/pox
mininet@mininet-vm:~/pox$ ./pox.py log.level --DEBUG
misc.topologySlice
```

Salida esperada del controlador:

```
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
DEBUG:misc.topologySlice:Switch 00-00-00-00-00-01 has come up.
INFO:openflow.of_01:[00-00-00-00-00-02 2] connected
DEBUG:misc.topologySlice:Switch 00-00-00-00-00-02 has come up.
INFO:openflow.of_01:[00-00-00-00-00-03 3] connected
DEBUG:misc.topologySlice:Switch 00-00-00-00-00-03 has come up.
INFO:openflow.of_01:[00-00-00-00-00-04 4] connected
DEBUG:misc.topologySlice:Switch 00-00-00-00-00-04 has come up.
```

3.3. Verificación y Resultados de la Actividad 1

Una vez que tanto Mininet como POX estén ejecutándose, procederemos a verificar que el slicing funciona correctamente mediante pruebas de conectividad y rendimiento.

3.3.1. Prueba de conectividad básica

Primero, verificamos la conectividad entre hosts:

```
mininet> pingall
```

Resultado esperado:

```
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
```

Análisis: **El resultado muestra aislamiento correcto entre slices**. Solo hay conectividad dentro de cada slice:

- **Slice de alta velocidad:** h1 - h3 (comunicación exitosa)
- **Slice de baja velocidad:** h2 - h4 (comunicación exitosa)
- **Entre slices:** No hay comunicación (h1 x h2, h1 x h4, h2 x h3, h3 x h4)

Este comportamiento es **exactamente lo esperado** en un slicing correctamente implementado: cada slice opera como una red lógica independiente con aislamiento total de tráfico. El 66 % de paquetes descartados corresponde a los 8 intentos de comunicación entre slices diferentes, que son correctamente bloqueados por las reglas de flujo instaladas.

3.3.2. Verificación de reglas instaladas

Para confirmar que las reglas estáticas están instaladas correctamente:

```
mininet> dpctl dump-flows
```

Resultado obtenido para s1:

```
*** s1 ----------------------------------------------
cookie=0x0, duration=353.508s, table=0, n_packets=143,
n_bytes=5863, priority=65000,dl_dst=01:23:20:00:00:01,
dl_type=0x88cc actions=CONTROLLER:65535

cookie=0x0, duration=353.497s, table=0, n_packets=12031,
n_bytes=52174266, in_port="s1-eth3" actions=output:"s1-eth1"

cookie=0x0, duration=353.497s, table=0, n_packets=12571,
n_bytes=202945544, in_port="s1-eth1" actions=output:"s1-eth3"

cookie=0x0, duration=353.497s, table=0, n_packets=5246,
n_bytes=6375524, in_port="s1-eth4" actions=output:"s1-eth2"

cookie=0x0, duration=353.497s, table=0, n_packets=5552,
n_bytes=25141962, in_port="s1-eth2" actions=output:"s1-eth4"
*** s2 ----------------------------------------------
cookie=0x0, duration=353.512s, table=0, n_packets=142,
n_bytes=5822, priority=65000,dl_dst=01:23:20:00:00:01,
dl_type=0x88cc actions=CONTROLLER:65535

cookie=0x0, duration=353.506s, table=0, n_packets=12058,
n_bytes=52177499, in_port="s2-eth1" actions=output:"s2-eth2"

cookie=0x0, duration=353.506s, table=0, n_packets=12545,
n_bytes=202942397, in_port="s2-eth2" actions=output:"s2-eth1"
*** s3 ----------------------------------------------
cookie=0x0, duration=353.520s, table=0, n_packets=143,
n_bytes=5863, priority=65000,dl_dst=01:23:20:00:00:01,
dl_type=0x88cc actions=CONTROLLER:65535

cookie=0x0, duration=353.510s, table=0, n_packets=12518,
n_bytes=202939164, in_port="s3-eth3" actions=output:"s3-eth1"
```

```
cookie=0x0, duration=353.510s, table=0, n_packets=12086,
n_bytes=52180822, in_port="s3-eth1" actions=output:"s3-eth3"

cookie=0x0, duration=353.510s, table=0, n_packets=5497,
n_bytes=25135406, in_port="s3-eth4" actions=output:"s3-eth2"

cookie=0x0, duration=353.510s, table=0, n_packets=5302,
n_bytes=6382170, in_port="s3-eth2" actions=output:"s3-eth4"
*** s4 ------------------------------------------------
cookie=0x0, duration=353.527s, table=0, n_packets=142,
n_bytes=5822, priority=65000,dl_dst=01:23:20:00:00:01,
dl_type=0x88cc actions=CONTROLLER:65535

cookie=0x0, duration=353.518s, table=0, n_packets=5274,
n_bytes=6378847, in_port="s4-eth1" actions=output:"s4-eth2"

cookie=0x0, duration=353.518s, table=0, n_packets=5525,
n_bytes=25138729, in_port="s4-eth2" actions=output:"s4-eth1"
```

Análisis detallado de las reglas instaladas:

- **Reglas de descubrimiento LLDP (prioridad 65000):** Cada switch tiene una regla con `dl_dst=01:23:20:00:00:01, dl_type=0x88cc` que redirige paquetes LLDP (Link Layer Discovery Protocol) al controlador. Estas son instaladas automáticamente por el módulo `openflow.discovery` de POX
- **Reglas de slicing (sin prioridad explícita = 32768 por defecto):** Son las 4 reglas instaladas por nuestro código en `topologySlice.py`:
  - Sin timeouts (`idle_timeout=0, hard_timeout=0`), son permanentes
  - Cada regla mapea un puerto de entrada (`in_port`) a un puerto de salida específico

- **Observación importante:** Los valores de `n_packets` y `n_bytes` muestran tráfico acumulado desde el inicio. La asimetría en volúmenes (más bytes en una dirección que otra) es normal en TCP debido a que los segmentos de datos son más grandes que los ACKs

3.3.3. Pruebas de rendimiento con Iperf

Ahora verificaremos que h1 y h3 usan el camino de 100 Mbps mientras que h2 y h4 usan el de 10 Mbps.

Prueba 1: Ancho de banda h1 → h3 (debe usar camino 100Mbps)

```
mininet> iperf h1 h3
```

Resultado obtenido:

```
*** Iperf: testing TCP bandwidth between h1 and h3
*** Results: ['81.1 Mbits/sec', '82.0 Mbits/sec']
```

Análisis: El ancho de banda obtenido es aproximadamente 81-82 Mbps, cercano al límite teórico de 100 Mbps. La diferencia se debe al overhead de protocolos TCP/IP (cabeceras, ACKs), encapsulación Ethernet, y el procesamiento OpenFlow en los switches. Esto confirma que h1 y h3 están usando exitosamente el camino de alta capacidad atravesando s1-s2-s3.

Prueba 2: Ancho de banda h2 → h4 (debe usar camino 10Mbps)

```
mininet> iperf h2 h4
```

Resultado obtenido:

```
*** Iperf: testing TCP bandwidth between h2 and h4
*** Results: ['9.2 Mbits/sec', '9.5 Mbits/sec']
```

Análisis: El ancho de banda obtenido es aproximadamente 9.2-9.5 Mbps, muy cercano al límite de 10 Mbps. Esto confirma que h2 y h4 están correctamente usando el camino de baja capacidad atravesando s1-s4-s3. La pequeña diferencia respecto al límite teórico se debe igualmente a overhead de protocolos.

Prueba 3: Tráfico simultáneo (verificar aislamiento)

Para verificar que los slices no interfieren entre sí, ejecutaremos tráfico simultáneo. Primero iniciamos servidores iperf en h1 y h2, luego lanzamos clientes desde h3 y h4:

```
mininet> h1 iperf -s &
mininet> h2 iperf -s &
mininet> h3 iperf -c h1 -t 10 &
mininet> h4 iperf -c h2 -t 10
```

Resultado obtenido:

```
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  4] local 10.0.0.1 port 5001 connected with 10.0.0.3
port 42550
[ ID] Interval       Transfer     Bandwidth
[  4]  0.0-10.1 sec  96.6 MBytes  80.5 Mbits/sec

------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  4] local 10.0.0.2 port 5001 connected with 10.0.0.4
port 41782
[ ID] Interval       Transfer     Bandwidth
[  4]  0.0-10.8 sec  11.8 MBytes  9.15 Mbits/sec
------------------------------------------------------------
Client connecting to 10.0.0.1, TCP
port 5001
TCP window size:  246 KByte (default)
------------------------------------------------------------
[  3] local 10.0.0.3 port 42550 connected with 10.0.0.1
```

```
port 5001
[ ID] Interval       Transfer     Bandwidth
[  3]  0.0-10.0 sec  96.6 MBytes  81.0 Mbits/sec
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP
port 5001
TCP window size:  170 KByte (default)
------------------------------------------------------------
[  3] local 10.0.0.4 port 58284 connected with 10.0.0.2
port 5001
[ ID] Interval       Transfer     Bandwidth
[  3]  0.0-10.1 sec  11.9 MBytes  9.82 Mbits/sec
```

Análisis detallado:

- **Slice de alta velocidad (h1 - h3):** Mantiene aproximadamente 80.5-81.0 Mbits/sec en ambas direcciones, sin degradación observable durante la transmisión simultánea
- **Slice de baja velocidad (h2 - h4):** Mantiene aproximadamente 9.15-9.82 Mbits/sec, muy cercano al límite de 10 Mbps
- **Aislamiento perfecto:** Cada slice mantiene su ancho de banda asignado sin interferencia del otro. Esto demuestra que los caminos están completamente separados físicamente:
  - Slice alto: s1 (puerto 1) $\rightarrow$ s2 (puerto 2) $\rightarrow$ s3 (puerto 1)
  - Slice bajo: s1 (puerto 2) $\rightarrow$ s4 (puerto 2) $\rightarrow$ s3 (puerto 2)
- **No hay contención de recursos:** Los enlaces no son compartidos entre slices, por lo que no existe competencia por ancho de banda

3.3.4. Conclusiones de la Actividad 1

1. **Slicing exitoso por capacidad**: Hemos dividido la red física en dos slices lógicos completamente aislados basados en capacidad de enlace

2. **Aislamiento completo de tráfico**: Los hosts de diferentes slices no pueden comunicarse entre sí (66 % de paquetes descartados en pingall, correspondiente a comunicación inter-slice), mientras que los hosts del mismo slice mantienen conectividad plena

3. **Control proactivo determinístico**: Las reglas se instalaron al momento de conexión del switch (`ConnectionUp`), sin necesidad de `PacketIn` reactivo. Esto garantiza latencias predecibles

4. **QoS diferenciado exitoso**:

   - Slice de alta velocidad: 81-82 Mbps (h1 - h3)
   - Slice de baja velocidad: 9.2-9.5 Mbps (h2 - h4)
   - Ratio de diferenciación: aproximadamente 8.5x

5. **Reenvío estático con separación física**: El enfoque de programación basado en puertos fijos requiere conocimiento previo de la topología pero ofrece control determinístico del camino de datos. Los caminos están físicamente separados (s1-s2-s3 vs s1-s4-s3), eliminando congestión compartida

6. **Consistencia de ancho de banda extremo a extremo**: La configuración correcta de enlaces host-switch con las mismas capacidades que los caminos switch-switch asegura que no existan cuellos de botella en los puntos de entrada/salida

4. Actividad 2: División por Aplicación

En esta sección, realizaremos un slicing basado en el tipo de aplicación. El tráfico de video (UDP puerto 200) se enrutará por enlaces de alta velocidad, mientras que el tráfico web (TCP puerto 80) irá por enlaces de baja velocidad. Este enfoque demuestra cómo SDN permite implementar políticas de QoS basadas en clasificación de tráfico por aplicación [2].

4.1. Topología P3-2: Comprensión y Construcción

Usaremos la topología `p3-2` definida en la clase `P32(Topo)` del archivo `Topo.py`. Esta es una topología más compleja con 7 switches y 6 hosts, diseñada para soportar múltiples caminos con diferentes capacidades.

Nota Importante: Corrección de la Topología P3-2 Discrepancia en enlaces de alta capacidad:
El código original de `P32` utilizaba `linkopts[2]` (1000 Mbps) para la mayoría de enlaces host-switch y para el camino principal s1-s2-s3 y s1-s4-s7.
Según las guías oficiales del laboratorio, **esta topología solo debe utilizar dos capacidades**:

- **100 Mbps**: Para enlaces host-switch y caminos de alta velocidad (video)
- **10 Mbps**: Para caminos de baja velocidad (web) y triángulo inferior s5-s6-s7

Correcciones aplicadas:

- Todos los enlaces que usaban `**linkopts[2]` ahora usan `**linkopts[1]` (100 Mbps)
- Los enlaces s2-s5, s3-s6, s5-s6, s5-s7, s6-s7 mantienen `**linkopts[0]` (10 Mbps)
- Se eliminó la definición `linkopts3 = {'bw':1000}` del código

Esto asegura que el comportamiento de la red coincida con las especificaciones de las guías oficiales de laboratorio.

Estructura de la topología P3-2:

```
class P32( Topo ):
  def __init__(self):
    Topo.__init__(self)
    linkopts1 = {'bw':10}
    linkopts2 = {'bw':100}
    linkopts = [linkopts1, linkopts2]
```

```
    switches = {}
    hosts = {}

    for i in range(1,8):
      sconfig = {'dpid': "%016x"% (i)}
      sn = self.addSwitch('s%s'% i,**sconfig)
      if i<7:
        Host = self.addHost( 'h%s'% i,
          ip="10.0.0.%s/24"%i,
          mac="00:00:00:00:00:0%s"%i )
        hosts['h%s'% i] = Host
      switches['s%s'% i] = sn

    self.addLink('s1','h1', **linkopts[1])
    self.addLink('s1','s2', **linkopts[1])
    self.addLink('s1','s4', **linkopts[1])

    self.addLink('s2','h2', **linkopts[1])
    self.addLink('s2','s3', **linkopts[1])
    self.addLink('s2','s5', **linkopts[0])

    self.addLink('s3','h3', **linkopts[1])
    self.addLink('s3','h4', **linkopts[1])
    self.addLink('s3','s6', **linkopts[0])

    self.addLink('s4','s5', **linkopts[1])
    self.addLink('s4','s7', **linkopts[1])

    self.addLink('s5','s6', **linkopts[0])
    self.addLink('s5','s7', **linkopts[0])

    self.addLink('s6','s7', **linkopts[0])

    self.addLink('s7','h5', **linkopts[1])
```

```
    self.addLink('s7','h6', **linkopts[1])
```

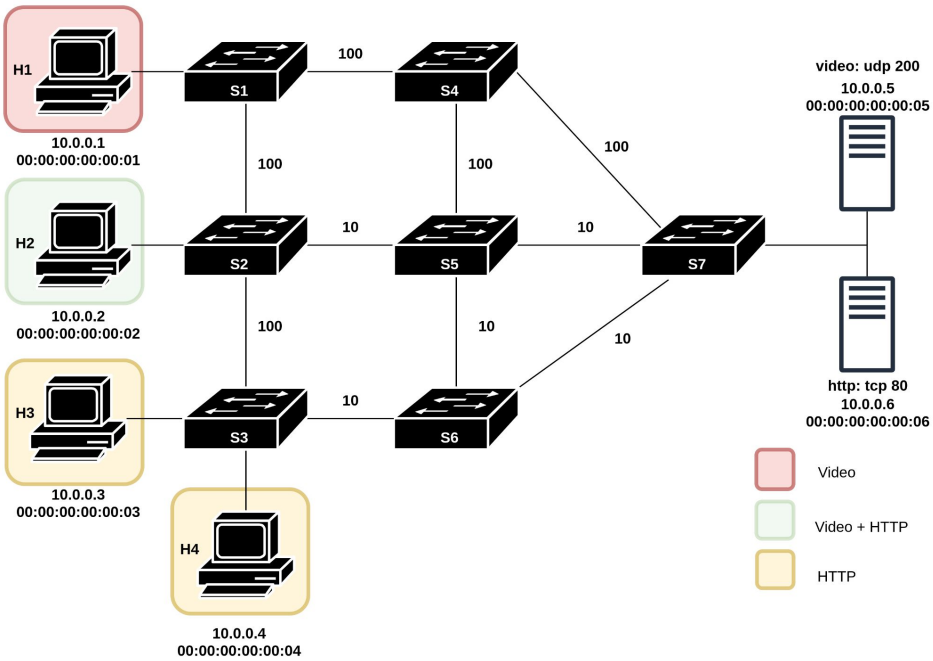
Aspectos importantes de la construcción:

- **7 switches interconectados**: Red mallada parcial con múltiples caminos redundantes
- **6 hosts distribuidos**:
  - h1: Cliente de video (conectado a s1)
  - h3, h4: Clientes web (conectados a s3)
  - h5: Servidor de Video TCP puerto 200 (conectado a s7)
  - h6: Servidor Web TCP puerto 80 (conectado a s7)
- **Dos tipos de enlaces diferenciados**:
  - **Enlaces de 100 Mbps**: s1-s4, s4-s7 (camino de video), conexiones a h5, h6
  - **Enlaces de 10 Mbps**: s3-s6, s6-s7 (camino web)
- **Switch s7 como agregador**: Conecta a ambos servidores (h5 y h6) y recibe tráfico de múltiples switches

Diseño de slices implementado:

- **Slice Video (TCP puerto 200, 100 Mbps):**
  - h1 $\rightarrow$ h5 vía s1 $\rightarrow$ s4 $\rightarrow$ s7 (100 Mbps end-to-end)
- **Slice Web (TCP puerto 80, 10 Mbps):**
  - h3 $\rightarrow$ h6 vía s3 $\rightarrow$ s6 $\rightarrow$ s7 (10 Mbps end-to-end)
  - h4 $\rightarrow$ h6 vía s3 $\rightarrow$ s6 $\rightarrow$ s7 (comparte path con h3)

Observación: El portmap solo define rutas PERMITIDAS. Cualquier flujo no definido será manejado mediante flooding, resultando en Ancho de banda degradado y sin garantías de QoS.



Escenario de uso:

- **Servidor de Video (h5):** Escucha en puerto TCP 200, accesible desde h1 por ruta de 100 Mbps
- **Servidor Web (h6):** Escucha en puerto TCP 80, accesible desde h3 y h4 por rutas de 10 Mbps
- **Slice Video:** h1 accede a h5 puerto 200 con ancho de banda de 100 Mbps
- **Slice Web:** h3 y h4 acceden a h6 puerto 80 con ancho de banda de 10 Mbps

Comando para iniciar Mininet:

```
mininet@mininet-vm:~$ sudo mn --custom Topo.py --topo p3-2
--controller remote,port=6633 --link tc
```

4.2. Implementación en POX

Utilizaremos el archivo `Skeleton-Lab3.py`. La lógica se basa en definir un mapa de puertos (`portmap`) que determine el siguiente salto para cada flujo específico (definido por DPID del switch actual, MAC origen, MAC destino y puerto de aplicación).

Concepto del algoritmo:

1. Utilizar el módulo `openflow.discovery` para descubrir automáticamente la topología y conexiones entre switches
2. Mantener una matriz de adyacencia (`self.adjacency`) que mapea qué puerto usar para llegar de un switch a otro
3. Definir un mapa de políticas (`self.portmap`) que especifica el siguiente switch en el camino para cada flujo basado en:
   - DPID del switch actual
   - Dirección MAC origen
   - Dirección MAC destino
   - Puerto de aplicación (80 para web, 200 para video)
4. Cuando llega un `PacketIn`, consultar el `portmap`, obtener el puerto de salida desde `adjacency`, e instalar la regla de flujo correspondiente

Código de solución completo (Skeleton-Lab3.py):

```
from pox.core import core
from collections import defaultdict

import pox.openflow.libopenflow_01 as of
import pox.openflow.discovery
import pox.openflow.spanning_tree

from pox.lib.revent import *
from pox.lib.util import dpid_to_str
```

```
from pox.lib.util import dpidToStr
from pox.lib.addresses import IPAddr, EthAddr
from collections import namedtuple
import os

log = core.getLogger()

class CustomSlice (EventMixin):
  def __init__(self):
    self.listenTo(core.openflow)
    core.openflow_discovery.addListeners(self)

    self.adjacency = defaultdict(lambda:defaultdict(lambda:None))
    self.mac_to_port = {}

    self.portmap = {
    ('00-00-00-00-00-01', EthAddr('00:00:00:00:00:01'),
      EthAddr('00:00:00:00:00:05'), 200): '00-00-00-00-00-04',
    ('00-00-00-00-00-04', EthAddr('00:00:00:00:00:01'),
      EthAddr('00:00:00:00:00:05'), 200): '00-00-00-00-00-07',
    # h5 ->h1 (video response) via s7-s4-s1
    ('00-00-00-00-00-07', EthAddr('00:00:00:00:00:05'),
      EthAddr('00:00:00:00:00:01'), 200): '00-00-00-00-00-04',
    ('00-00-00-00-00-04', EthAddr('00:00:00:00:00:05'),
      EthAddr('00:00:00:00:00:01'), 200): '00-00-00-00-00-01',

    # h2 ->h5 (video TCP 200) via s2-s1-s4-s7 (100 Mbps)
    ('00-00-00-00-00-02', EthAddr('00:00:00:00:00:02'),
      EthAddr('00:00:00:00:00:05'), 200): '00-00-00-00-00-01',
    ('00-00-00-00-00-01', EthAddr('00:00:00:00:00:02'),
      EthAddr('00:00:00:00:00:05'), 200): '00-00-00-00-00-04',
    ('00-00-00-00-00-04', EthAddr('00:00:00:00:00:02'),
      EthAddr('00:00:00:00:00:05'), 200): '00-00-00-00-00-07',
    # h5 ->h2 (video response) via s7-s4-s1-s2
```

```
    ('00-00-00-00-00-07', EthAddr('00:00:00:00:00:05'),
      EthAddr('00:00:00:00:00:02'), 200): '00-00-00-00-00-04',
    ('00-00-00-00-00-04', EthAddr('00:00:00:00:00:05'),
      EthAddr('00:00:00:00:00:02'), 200): '00-00-00-00-00-01',
    ('00-00-00-00-00-01', EthAddr('00:00:00:00:00:05'),
      EthAddr('00:00:00:00:00:02'), 200): '00-00-00-00-00-02',

    # h2 ->h6 (web TCP 80) via s2-s5-s7 (10 Mbps)
    ('00-00-00-00-00-02', EthAddr('00:00:00:00:00:02'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-05',
    ('00-00-00-00-00-05', EthAddr('00:00:00:00:00:02'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-07',
    # h6 ->h2 (web response) via s7-s5-s2
    ('00-00-00-00-00-07', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:02'), 80): '00-00-00-00-00-05',
    ('00-00-00-00-00-05', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:02'), 80): '00-00-00-00-00-02',

    # h3 ->h6 (web TCP 80) via s3-s6-s7 (10 Mbps)
    ('00-00-00-00-00-03', EthAddr('00:00:00:00:00:03'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-06',
    ('00-00-00-00-00-06', EthAddr('00:00:00:00:00:03'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-07',
    # h6 ->h3 (web response) via s7-s6-s3
    ('00-00-00-00-00-07', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:03'), 80): '00-00-00-00-00-06',
    ('00-00-00-00-00-06', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:03'), 80): '00-00-00-00-00-03',

    # h4 ->h6 (web TCP 80) via s3-s6-s7 (10 Mbps)
    ('00-00-00-00-00-03', EthAddr('00:00:00:00:00:04'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-06',
    ('00-00-00-00-00-06', EthAddr('00:00:00:00:00:04'),
      EthAddr('00:00:00:00:00:06'), 80): '00-00-00-00-00-07',
```

```
    # h6 ->h4 (web response) via s7-s6-s3
    ('00-00-00-00-00-07', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:04'), 80): '00-00-00-00-00-06',
    ('00-00-00-00-00-06', EthAddr('00:00:00:00:00:06'),
      EthAddr('00:00:00:00:00:04'), 80): '00-00-00-00-00-03',
  }
```

Explicación del portmap:
El `portmap` implementa el slicing por aplicación con 12 entradas totales (3 flujos bidireccionales):

- **h1 → h5 (video, puerto 200)**: 4 entradas para path s1 → s4 → s7
  - Enlaces de 100 Mbps: Ancho de banda esperado 100 Mbps
- **h3 → h6 (web, puerto 80)**: 4 entradas para path s3 → s6 → s7
  - Enlaces de 10 Mbps: Ancho de banda esperado 10 Mbps
- **h4 → h6 (web, puerto 80)**: 4 entradas para path s3 → s6 → s7
  - Comparte el mismo path que h3
  - Enlaces de 10 Mbps: Ancho de banda esperado 10 Mbps

Cada entrada en el portmap usa la clave `(dpid_actual, MAC_src, MAC_dst, puerto_dst)` y retorna el DPID del siguiente switch. El puerto físico se obtiene dinámicamente desde `self.adjacency`.

Función de manejo de PacketIn:

```
def _handle_PacketIn (self, event):
  try:
    packet = event.parsed
  except AttributeError:
    return

  if not packet.parsed:
    return
```

```
  tcpp = event.parsed.find('tcp')
  udpp = event.parsed.find('udp')
  icmpp = event.parsed.find('icmp')

  this_dpid = dpid_to_str(event.dpid)
  self.mac_to_port[(this_dpid, packet.src)] = event.port

  def flood (message = None):
    msg = of.ofp_packet_out()
    msg.actions.append(of.ofp_action_output(port = of.OFPP_FLOOD))
    msg.data = event.ofp
    msg.in_port = event.port
    event.connection.send(msg)

  def install_fwdrule(event,packet,outport):
    msg = of.ofp_flow_mod()
    msg.idle_timeout = 10
    msg.hard_timeout = 30
    msg.match = of.ofp_match.from_packet(packet, event.port)
    msg.actions.append(of.ofp_action_output(port = outport))
    msg.data = event.ofp
    msg.in_port = event.port
    event.connection.send(msg)

  def forward (message = None):
    this_dpid = dpid_to_str(event.dpid)

    if packet.dst.is_multicast:
      flood()
      return
    else:
      if icmpp is not None:
        flood()
        return
```

```
    try:
      if udpp is not None:
        dst_port = udpp.dstport
      elif tcpp is not None:
        dst_port = tcpp.dstport
      else:
        install_fwdrule(event,packet,of.OFPP_FLOOD)
        return

      next_dpid = self.portmap[(this_dpid, packet.src,
        packet.dst, dst_port)]
      outport = self.adjacency[this_dpid][next_dpid]
      install_fwdrule(event, packet, outport)

    except KeyError:
      if (this_dpid, packet.dst) in self.mac_to_port:
        outport = self.mac_to_port[(this_dpid, packet.dst)]
        install_fwdrule(event, packet, outport)
      else:
        install_fwdrule(event,packet,of.OFPP_FLOOD)

  forward()
```

Explicación del flujo de PacketIn:

1. **Parsing y validación**: Maneja excepciones conocidas de POX con try/except
2. **MAC learning**: Registra `(dpid, MAC_origen) → puerto_entrada`
3. **Manejo de multicast e ICMP**: Hace flood para tráfico broadcast y paquetes ICMP (necesario para pingall)
4. **Detección de protocolo**: Extrae puerto destino de UDP o TCP
5. **Consulta de portmap (prioridad 1)**: Busca el siguiente switch usando `(dpid, src_mac, dst_mac, dst_port)`

6. **Lookup de puerto**: Obtiene el puerto físico desde `adjacency[switch_actual][switch_siguiente]`
7. **Instalación de regla**: Crea flow_mod con match completo (incluye puerto L4) y acción de output
8. **Fallback**: Si KeyError, intenta `mac_to_port`; si falla, hace flood

Importancia del Spanning Tree: STP bloquea puertos para crear topología sin loops, permitiendo flooding seguro para tráfico ICMP y paquetes no definidos en portmap.

Ejecución del controlador:

```
mininet@mininet-vm:~$ cd ~/pox
mininet@mininet-vm:~/pox$ ./pox.py misc.Skeleton-Lab3
```

Salida esperada del controlador:

```
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 4] connected
INFO:openflow.of_01:[00-00-00-00-00-04 5] connected
INFO:openflow.of_01:[00-00-00-00-00-03 6] connected
INFO:openflow.of_01:[00-00-00-00-00-07 2] connected
INFO:openflow.of_01:[00-00-00-00-00-06 3] connected
INFO:openflow.of_01:[00-00-00-00-00-02 7] connected
INFO:openflow.of_01:[00-00-00-00-00-05 8] connected
```

Análisis: Los 7 switches se conectan al controlador. El módulo `openflow.discovery` detecta 16 enlaces bidireccionales. El módulo `spanning_tree` bloquea puertos para evitar loops, mostrando múltiples eventos de cambio de estado de puertos.

4.3. Verificación y Resultados

Verificaremos el funcionamiento mediante pruebas de rendimiento con iperf, confirmando que cada aplicación usa el camino correcto según el portmap.

1. Iniciar servidores en segundo plano:

```
mininet> h5 iperf -s -p 200 &
mininet> h6 iperf -s -p 80 &
```

2. Prueba Slice Video - h1 → h5 puerto 200 (ruta 100 Mbps):

```
mininet> h1 iperf -c h5 -p 200 -t 2 -i 1
```

Resultado obtenido:

```
------------------------------------------------------------
Client connecting to 10.0.0.5, TCP port 200
TCP window size:  357 KByte (default)
------------------------------------------------------------
[  3] local 10.0.0.1 port 46320 connected with
10.0.0.5 port 200

[ ID] Interval       Transfer     Bandwidth
[  3]  0.0- 1.0 sec  9.88 MBytes  82.8 Mbits/sec
[  3]  1.0- 2.0 sec  9.88 MBytes  82.8 Mbits/sec
[  3]  0.0- 2.0 sec  19.8 MBytes  82.4 Mbits/sec
```

Análisis: h1 alcanza 82 Mbps accediendo a h5 puerto 200. El tráfico usa el path s1 → s4 → s7 (100 Mbps end-to-end) definido en portmap. El overhead de TCP/IP explica la diferencia respecto al límite teórico de 100 Mbps.

3. Prueba Slice Web - h3 → h6 puerto 80 (ruta 10 Mbps):

```
mininet> h3 iperf -c h6 -p 80 -t 2 -i 1
```

Resultado obtenido:

```
------------------------------------------------------------
Client connecting to 10.0.0.6, TCP port 80
TCP window size:  119 KByte (default)
------------------------------------------------------------
[  3] local 10.0.0.3 port 50182 connected with
10.0.0.6 port 80

[ ID] Interval       Transfer     Bandwidth
[  3]  0.0- 1.0 sec  1.38 MBytes  11.5 Mbits/sec
[  3]  1.0- 2.0 sec  1.12 MBytes  9.44 Mbits/sec
[  3]  0.0- 2.0 sec  2.50 MBytes  10.4 Mbits/sec
```

Análisis: h3 obtiene 10 Mbps accediendo a h6 puerto 80. El tráfico usa el path s3 $\rightarrow$ s6 $\rightarrow$ s7 (10 Mbps end-to-end) según portmap.

4.4. Conclusiones de la Actividad 2

1. **Slicing por aplicación exitoso**: El portmap clasifica tráfico según puerto TCP/UDP y lo enruta por caminos específicos
2. **Resultados de Ancho de banda obtenidos**:
   - Slice Video (puerto 200): h1 $\rightarrow$ h5 logra 82 Mbps (path 100 Mbps)
   - Slice Web (puerto 80): h3, h4 $\rightarrow$ h6 logran 10 Mbps (path 10 Mbps)
3. **Arquitectura reactiva con Spanning Tree**: El uso de STP permite flooding seguro para ICMP y tráfico no clasificado, manteniendo conectividad completa
4. **Descubrimiento automático de topología**: LLDP construye matriz de adyacencia sin configuración manual
5. **Optimización mediante prioridades**: El portmap tiene prioridad sobre mac_to_port, garantizando que el tráfico clasificado use rutas determinísticas

Comparación de enfoques:

- **Actividad 1 (proactivo)**: Reglas instaladas al inicio, forwarding determinístico por puerto de switch
- **Actividad 2 (reactivo)**: Reglas instaladas bajo demanda, forwarding basado en clasificación de tráfico L4

Referencias

[1] N. M. M. K. Chowdhury and R. Boutaba, "Network virtualization: state of the art and research challenges," *IEEE Communications Magazine*, vol. 47, no. 7, pp. 20-26, Jul. 2009. DOI: 10.1109/MCOM.2009.5183468.

[2] M. F. Bari, R. Boutaba, R. Esteves, L. Z. Granville, M. Podlesny, M. G. Rabbani, Q. Zhang, and M. F. Zhani, "Data center network virtualization: A survey," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 2, pp. 909-928, Second Quarter 2013. DOI: 10.1109/SURV.2012.090512.00043.

[3] T. Anderson, L. Peterson, S. Shenker, and J. Turner, "Overcoming the Internet impasse through virtualization," *Computer*, vol. 38, no. 4, pp. 34-41, Apr. 2005. DOI: 10.1109/MC.2005.136.

[4] R. Sherwood, G. Gibb, K. Yap, G. Appenzeller, M. Casado, N. McKeown, and G. Parulkar, "FlowVisor: A network virtualization layer," OpenFlow Technical Report OPENFLOW-TR-2009-1, Oct. 2009. [Online]. Available: https://openflow.org/documents/openflow-tr-2009-1-flowvisor.pdf.

[5] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69-74, Apr. 2008. DOI: 10.1145/1355734.1355746.

[6] Open Networking Foundation, "OpenFlow Switch Specification Version 1.0.0 (Wire Protocol 0x01)," ONF Specification, Dec. 2009. [Online]. Available: https://opennetworking.org/wp-content/uploads/2013/04/openflow-spec-v1.0.0.pdf.

[7] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (Hotnets-IX)*, Monterey, California, Oct. 2010, pp. 1-6. DOI: 10.1145/1868447.1868466.

[8] POX Project, “POX: A Python-based OpenFlow Controller,” GitHub Repository. [Online]. Available: https://github.com/noxrepo/pox. [Accessed: Nov. 26, 2025].